UD 6. Utilización del modelo de objetos del documento (DOM)

Índice

6.1. Introducción DOM. Modelo Objeto Documento 3
6.1.1. Acceso relativo a los nodos 3
6.1.2. Tipos de nodos 4
6.1.3. Atributos 5
6.2. Acceso directo a nodos 6
6.2.1. getElementsByTagName() 6
6.2.2. getElementsByName() 7
6.2.3. getElementById() 7
6.2.4. Ejercicios 7
6.3. Detección Carga DOM 9
6.3.1 Evento Load 9
6.3.2 Propiedad readState: Eventos readstatechange y DOMContentLoaded 9
6.4 Modificar el DOM 11
6.5 Tablas 12
6.5.1 Ejercicios 15
6.6. Ejercicio de Ampliación 17

Autor: Roberto Marín

Módulo: Desarrollo Web Entorno Cliente (DAW2)

Año: 2023/2024

COPYRIGHT:

Reconocimiento-NoComercial-CompartirIgual. CC BY-NC-SA
Esta licencia permite a otros entremezclar, ajustar y construir a partir de su obra con fines no comerciales, siempre y cuando le reconozcan la autoría y sus nuevas creaciones estén bajo una licencia con los mismos términos.

No se permite el uso de este material en ninguna convocatoria oficial de oposiciones.

6.1. Introducción DOM. Modelo Objeto Documento

Las posibilidades teóricas de DOM son mucho más avanzadas de las que se pueden utilizar en la práctica para desarrollar aplicaciones web. El motivo es que el uso de DOM siempre está limitado por las posibilidades que ofrece cada navegador. Mientras que algunos navegadores como Chrome, Edge, Firefox y Safari implementan DOM de nivel 1, 2 y 3), otros navegadores como Internet Explorer (versión 7 y anteriores) ni siquiera son capaces de ofrecer una implementación completa de DOM nivel 1.

Cuando se utiliza DOM en páginas HTML, el nodo raíz de todos los demás se define en el objeto HTMLDocument. Además, se crean objetos de tipo HTMLElement por cada nodo de tipo Element del árbol DOM.

6.1.1. Acceso relativo a los nodos

A continuación se muestra la página HTML básica que se va a emplear en todos los siguientes ejemplos:

```
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>Aprendiendo DOM</title>
</head>
<body>
<script type="text/javascript">
</script>
    <p>Aprendiendo DOM</p>
    <p>DOM es sencillo de aprender</p>
    <p>Ademas, DOM es muy potente</p>
  </body>
</html>
```

La operación básica consiste en obtener el objeto que representa el elemento raíz de la página:

```
let objeto_html = document.documentElement;
```

Después de ejecutar la instrucción anterior, la variable objeto_html contiene un objeto de tipo HTMLElement y que representa el elemento <html> de la página web. Según el árbol de nodos DOM, desde el nodo <html> derivan dos nodos del mismo nivel jerárquico: <head> y <body>.

Utilizando los métodos proporcionados por DOM, es sencillo obtener los elementos <head> y <body>. En primer lugar, los dos nodos se pueden obtener como el primer y el último nodo hijo del elemento <html>:

```
  let objeto_head = objeto_html.firstChild;
  let objeto_body = objeto_html.lastChild;
```

Otra forma directa de obtener los dos nodos consiste en utilizar la propiedad childNodes del elemento <html>:

```
//let objeto_head = objeto_html.childNodes[0];
//let objeto_body = objeto_html.childNodes[1];
```

Si se desconoce el número de nodos hijo que dispone un nodo, se puede emplear la propiedad length de childNodes:

```
let numeroDescendientes = objeto_html.childNodes.length;
```

Además, el DOM de HTML permite acceder directamente al elemento <body> utilizando el atajo document.body:

```
let objeto_body = document.body;
```

Además de las propiedades anteriores, existen otras propiedades como previousSibling y parentNode que se pueden utilizar para acceder a un nodo a partir de otro. Utilizando estas propiedades, se pueden comprobar las siguientes igualdades:

```
console.log(objeto_head.parentNode == objeto_html); //true
console.log(objeto_body.parentNode == objeto_html); //true
console.log(objeto_body.previousSibling == objeto_head); //false
console.log(objeto_head.nextSibling == objeto_body); //false
console.log(objeto_head.ownerDocument == document); //true
```

6.1.2. Tipos de nodos

Una operación común en muchas aplicaciones consiste en comprobar el tipo de nodo, que se obtiene de forma directa mediante la propiedad nodeType:

```
console.log(document.nodeType); // 9
console.log(document.documentElement.nodeType); // 1
```

En el primer caso, el valor 9 es igual al definido en la constante Node.DOCUMENT_NODE. En el segundo ejemplo, el valor 1 coincide con la constante Node.ELEMENT_NODE. Afortunadamente, no es necesario memorizar los valores numéricos de los tipos de nodos, ya que se pueden emplear las constantes predefinidas:

```
console.log(document.nodeType == Node.DOCUMENT_NODE); // true
console.log(document.documentElement.nodeType == Node.ELEMENT_NODE); // true
```

NOTA: El único navegador que no soporta las constantes predefinidas es Internet Explorer 7 y sus versiones anteriores, por lo que si se quieren utilizar es necesario definirlas de forma explícita:

```
if(typeof Node == "undefined") {
  var Node = {
    ELEMENT_NODE: 1,
    ATTRIBUTE_NODE: 2,
    TEXT_NODE: 3,
    CDATA_SECTION_NODE: 4,
    ENTITY_REFERENCE_NODE: 5,
    ENTITY_NODE: 6,
    PROCESSING_INSTRUCTION_NODE: 7,
    COMMENT_NODE: 8,
    DOCUMENT_NODE: 9,
    DOCUMENT_TYPE_NODE: 10,
    DOCUMENT_FRAGMENT_NODE: 11,
    NOTATION_NODE: 12
  }
}
```

El código anterior comprueba si el navegador en el que se está ejecutando tiene definido el objeto Node. Si este objeto no está definido, se trata del navegador Internet Explorer 7 o alguna versión anterior, por lo que se crea un nuevo objeto llamado Node y se le incluyen como propiedades todas las constantes definidas por DOM.

6.1.3. Atributos

Además del tipo de etiqueta HTML y su contenido de texto, DOM permite el acceso directo a todos los atributos de cada etiqueta. Para ello, los nodos de tipo Element contienen la propiedad attributes, que permite acceder a todos los atributos de cada elemento. Aunque técnicamente la propiedad attributes es de tipo NamedNodeMap, sus elementos se pueden acceder como si fuera un array. DOM proporciona diversos métodos para tratar con los atributos:

- getNamedItem(nombre), devuelve el nodo cuya propiedad nodeName contenga el valor nombre.
- removeNamedItem(nombre), elimina el nodo cuya propiedad nodeName coincida con el valor nombre.
- setNamedItem(nodo), añade el nodo a la lista attributes, indexándolo según su propiedad nodeName.
- item(posicion), devuelve el nodo que se encuentra en la posición indicada por el valor numérico posicion.

Los métodos anteriores devuelven un nodo de tipo Attr, por lo que no devuelven directamente el valor del atributo. Utilizando estos métodos, es posible procesar y modificar fácilmente los atributos de los elementos HTML:

```
<body>
  <p id="introduccion" style="color: blue">Parrafo de prueba</p>
</body>

<script type="text/javascript">
let p = document.getElementById("introduccion");
let elId = p.attributes.getNamedItem("id").nodeValue; // elId = "introduccion"
let elId = p.attributes.item(0).nodeValue;            // elId = "introduccion"
p.attributes.getNamedItem("id").nodeValue = "preintroduccion";

let atributo = document.createAttribute("lang"); // atributo lang
atributo.nodeValue = "es";                           // lang="es"
let p = document.getElementById("introduccion");
p.attributes.setNamedItem(atributo);                 //

console.log(elId);
console.log(p);
console.log(atributo);
</script>
```

Afortunadamente, DOM proporciona otros métodos que permiten el acceso y la modificación de los atributos de forma más directa:

- getAttribute(nombre), es igual a attributes.getNamedItem(nombre).
- setAttribute(nombre,valor), igual a attributes.getNamedItem(nombre).value = valor.
- removeAttribute(nombre), igual a attributes.removeNamedItem(nombre).

De esta forma, el ejemplo anterior se puede reescribir utilizando los nuevos métodos:

```
<body>
  <p id="introduccion" style="color: blue">Parrafo de prueba</p>
</body>

<script type="text/javascript">
```

```
let p = document.getElementById("introduccion");
let elId = p.getAttribute("id"); // elId = "introduccion"
p.setAttribute("id", "preintroduccion");

console.log(elId);
console.log(p);
</script>
```

6.2. Acceso directo a nodos

6.2.1. getElementsByTagName()

Como sucede con todas las funciones que proporciona DOM, la función getElementsByTagName() tiene un nombre muy largo, pero que lo hace muy clarificador.

La función getElementsByTagName(nombreEtiqueta) obtiene todos los elementos de la página XHTML cuya **etiqueta** sea igual que el parámetro que se le pasa a la función.

El siguiente ejemplo muestra cómo obtener todos los párrafos de una página XHTML:

```
let parrafos = document.getElementsByTagName("p");
```

El valor que se indica delante del nombre de la función (en este caso, document) es el nodo a partir del cual se realiza la búsqueda de los elementos. En este caso, como se quieren obtener todos los párrafos de la página, se utiliza el valor document como punto de partida de la búsqueda.

El valor que devuelve la función es un objeto NodeList con todos los elementos que cumplen la condición de que su etiqueta coincide con el parámetro proporcionado. El valor devuelto es un array de nodos DOM, no un array de cadenas de texto o un array de objetos normales. Por lo tanto, se debe procesar cada valor del array de la forma que se muestra en las siguientes secciones.

De este modo, se puede obtener el primer párrafo de la página de la siguiente manera:

```
let primerParrafo = parrafos[0];
```

De la misma forma, se podrían recorrer todos los párrafos de la página con el siguiente código:

```
for(var i=0; i<parrafos.length; i++) {
var parrafo = parrafos[i];
}
```

La función getElementsByTagName() se puede aplicar de forma recursiva sobre cada uno de los nodos devueltos por la función. En el siguiente ejemplo, se obtienen todos los enlaces del primer párrafo de la página:

```
let parrafos = document.getElementsByTagName("p");
let primerParrafo = parrafos[0];
let enlaces = primerParrafo.getElementsByTagName("a");
```

6.2.2. getElementsByName()

La función getElementsByName() es similar a la anterior, pero en este caso se buscan los elementos cuyo atributo name sea igual al parámetro proporcionado. En el siguiente ejemplo, se obtiene directamente el único párrafo con el nombre indicado:

```
let parrafoEspecial = document.getElementsByName("especial");
```

```
<p name="prueba">...</p>
<p name="especial">...</p>
<p>...</p>
```

Normalmente el atributo name es único para los elementos HTML que lo definen, por lo que es un método muy práctico para acceder directamente al nodo deseado. En el caso de los elementos HTML radiobutton, el atributo name es común a todos los radiobutton que están relacionados, por lo que la función devuelve un array con los elementos.

6.2.3. getElementById()

La función getElementById() es la más utilizada cuando se desarrollan aplicaciones web dinámicas. Se trata de la función preferida para acceder directamente a un nodo y poder leer o modificar sus propiedades.

La función getElementById() devuelve el elemento XHTML cuyo atributo id coincide con el parámetro indicado en la función. Como el atributo id debe ser único para cada elemento de una misma página, la función devuelve únicamente el nodo deseado.

```
let cabecera = document.getElementById("cabecera");
```

```
<div id="cabecera">
  <a href="/" id="logo">...</a>
</div>
```

6.2.4. Ejercicios

Árbol Dom 1

A partir de la página Web proporcionada y utilizando las funciones DOM, mostrar por pantalla la siguiente información:

1. Número de enlaces de la página
2. Dirección a la que enlaza el penúltimo enlace
3. Numero de enlaces que enlazan a http://prueba
4. Número de enlaces del tercer párrafo

Árbol Dom 2:

A partir de la página Web proporcionada y utilizando las funciones DOM:

1. Se debe modificar el protocolo de todas las direcciones de los enlaces. De esta forma, si un enlace apuntaba a http://prueba, ahora debe apuntar a https://prueba

2. Los párrafos de la página cuyo atributo class es igual a "importante" deben modificarlo por "resaltado". El resto de párrafos deben incluir un atributo class igual a "normal".

3. A los enlaces de la página cuyo atributo class sea igual a "importante", se les añade un atributo "name" con un valor generado automáticamente y que sea igual a "importante"+i, donde i es un valor numérico cuyo valor inicial es 0 para el primer enlace.

Árbol Dom 3:

Mediante el empleo de un script modifique el color de texto con un doble checkbox de un documento que esté contenido en:

```
<div id= "elementos Tag">
<div id ="divTexto">
Lorem lorem lorem……
…… Lorem lorem
</div>
</div>
```

Nota: Meter en un Form

Árbol Dom 4:

Crea un input de tipo text, y mediante un único botón crea una función cuya propiedad style del elemento HTML, oculte y muestre dicho input.
Ayuda: Document.getElementBtId().style

6.3. Detección Carga DOM

6.3.1 Evento Load

Dicho evento del objeto Window, permite identificar cuando se han cargado todos los recursos del documento.
Ejemplo:

```
window.onload = function(){
  alert('Documento cargado');}
```

6.3.2 Propiedad readState: Eventos readstatechange y DOMContentLoaded

Hay 3 estados posibles diferentes:

- Loading. El documento se está cargando (el .html archivo se está descargando / analizando)

  Se descarga el archivo index.html (este es el valor predeterminado si no se especifica nada) del servidor y se analiza. Si ha incluido scripts, también se descargan, analizan y ejecutan. Una vez que se carga el archivo HTML, readyState cambia a interactive, y podemos detectar esto usando el evento readystatechange.

- Interactive. El DOM está cargado y accesible. Sin embargo, recursos como imágenes, hojas de estilo y archivos JS no han terminado de descargarse / cargarse / analizarse.

  Significa que DOM está cargado, pero los recursos se están descargando.

  El evento DOMContentLoaded también se activa cuando el readyState cambia de loading a interactive.

- Complete. El documento y todos los recursos como imágenes / hojas de estilo han terminado de cargarse.

  Cuando readyState se cambia a complete, significa que el documento tiene analizado y cargado todos los recursos adicionales como pueden ser CSS, imágenes y JS.

Árbol Dom 5:
Detecte los 3 estados posibles de un documento con la propiedad readyStateChange mediante la apertura de una página Web. Emplea avisos por pantalla en cada uno de los pasos siguientes.
- loading: no carga de página ni de recursos.
- interactive: carga de página pero no de recursos.
- complete: carga de página y recursos.

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Carga Aplicación Web</title>
</head>
<body>
  <script>
    console.log("no carga de página ni de recursos....", document.readyState);
    document.addEventListener('readystatechange',function (ev) {
      console.log("statechange", document.readyState);
    })
    document.addEventListener("DOMContentLoaded", function(ev){
       console.log("DOM CARGADO", ev);
     });
  </script>
  <script>
    console.log("Este es un script");
  </script>
</body>
</html>
```

6.4 Modificar el DOM

Los nodos del DOM de un documento HTML serán del tipo elemento o de tipo texto coincidentes con los diversos tags (etiquetas) y el texto que encierran. Además un tag puede contener a otros tags que se convertirán así en hijos del primero.
A la hora de generar partes de código HTML mediante JavaScript tendremos dos opciones:

1. Usar innerHTML y pasar todo el contenido como una cadena de texto.
2. Usar las herramientas específicas del DOM para creación de nodos.

Supongamos que tenemos el siguiente código HTML:

```
<div id="d1">
<p>Este es el primer parrafo</p>
</div>
```

Y tenemos que introducir un segundo párrafo.

Con la primera opción haríamos de innerHTML:

```
document.getElementById("d1").innerHTML += "<p>Este es el segundo párrafo</p>"
```

Y con la segunda opción usando DOM, podremos crear elementos de manera dinámica:

```
let d=document.getElementById("d1");
let  p=document.createElement("p");
let  t=document.createTextNode("Este es el segundo párrafo");
p.appendChild(t);
d.appendChild(p);
```

En el ejemplo "**Arbol Dom 6**" añadiremos elementos mediante las dos formas y después accederemos mediante las funciones del DOM.
Como podemos ver ambas formas funcionan. El usar una o la otra dependerá del problema concreto.

NOTA: Con el método innerHTML es mas conciso y rápido de usar pero precisa ser cuidadoso para no arruinar la estructura del DOM existente. Este hecho unido a los posibles vectores de ataque XSS, hacen que se recomiende usar la opción DOM.

```
let d=document.getElementById("dni");
d.innerHTML+="<p>El DNI es valido</p>"
d.textContent += "El DNI es valido"; //carga mas rápido y evita ataques XSS
```

6.5 Tablas

La etiqueta <table> encierra todas las filas y columnas de la tabla. Las etiquetas <tr> (del inglés *"table row"*) definen cada fila de la tabla y encierran todas las columnas. Por último, la etiqueta <td> (del inglés *"table data cell"*) define cada una de las columnas de las filas, aunque realmente HTML no define columnas sino *celdas de datos*.

Al definir una tabla, se debe pensar en primer lugar en las filas que la forman y a continuación en las columnas. El motivo es que HTML procesa primero las filas y por eso las etiquetas <tr> aparecen antes que las etiquetas <td>.

Etiqueta	<table>
Atributos comunes	básicos, internacionalización y eventos
Atributos propios	• `summary = "texto"` - Permite describir el contenido de la tabla (lo utilizan los buscadores y las personas discapacitadas)
Tipo de elemento	Bloque
Descripción	Se emplea para definir tablas de datos
Etiqueta	<tr>
Atributos comunes	básicos, internacionalización y eventos
Atributos propios	-
Tipo de elemento	Bloque
Descripción	Se emplea para definir cada fila de las tablas de datos

Etiqueta	<td>
Atributos comunes	básicos, internacionalización y eventos
Atributos propios	• `abbr = "texto"` - Permite describir el contenido de la celda (empleado sobre todo en los navegadores utilizados por personas discapacitadas) • `headers = "lista_de_id"` - Indica las celdas que actúan como cabeceras para esta celda (los títulos de las columnas y filas). Se indica como una lista de valores del atributo "id" de celdas • `scope = "col, row, colgroup, rowgroup"` - Indica las celdas para las que esta celda será su cabecera. Ej: scope="col" indica que esta celda es la cabecera de todas las demás celdas que están en la misma columna • `colspan = "numero"` - Número de columnas que ocupa esta celda • `rowspan = "numero"` - Número de filas que ocupa esta celda
Tipo de elemento	Bloque
Descripción	Se emplea para definir cada una de las celdas que forman las filas de una tabla, es decir, las columnas de la tabla

De todos los atributos disponibles para las celdas, los más utilizados son `rowspan` y `colspan`, que se emplean para construir tablas complejas como las que se ven más adelante. Entre los demás atributos, sólo se utiliza de forma habitual el atributo `scope`, sobre todo con las celdas de cabecera que se ven a continuación.

Normalmente, algunas de las celdas de la tabla se utilizan como cabecera de las demás celdas de la fila o de la columna. En este caso, HTML define la etiqueta `<th>` (del inglés *"table header cell"*) para indicar que una celda es cabecera de otras celdas.

Etiqueta	<th>
Atributos comunes	básicos, internacionalización y eventos
Atributos propios	• `abbr = "texto"` - Permite describir el contenido de la celda (empleado sobre todo en los navegadores utilizados por personas discapacitadas)

Etiqueta	<th>
	• `headers = "lista_de_id"` - Indica las celdas que actúan como cabeceras para esta celda (los títulos de las columnas y filas). Se indica como una lista de ID de celdas • `scope = "col, row, colgroup, rowgroup"` - Indica las celdas para las que esta celda será su cabecera. Ej: scope="col" indica que esta celda es la cabecera de todas las demás celdas que están en la misma columna • `colspan = "numero"` - Número de columnas que ocupa esta celda • `rowspan = "numero"` - Número de filas que ocupa esta celda
Tipo de elemento	Bloque
Descripción	Se emplea para definir las celdas que son cabecera de una fila o de una columna de la tabla

Los atributos de la etiqueta `<th>` son idénticos que los atributos definidos para la etiqueta `<td>`. En este caso, el atributo más utilizado es `scope`, que permite indicar si la celda es cabecera de la fila o de la columna (`<th scope="row">` y `<th scope="col">` respectivamente).

Por otra parte, HTML define la etiqueta `<caption>` para establecer la leyenda o título de una tabla. La etiqueta debe colocarse inmediatamente después de la etiqueta `<table>` y cada tabla sólo puede incluir una etiqueta `<caption>`.

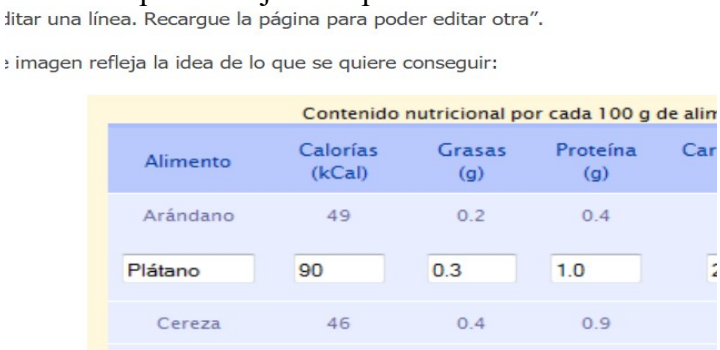
Etiqueta	<caption>
Atributos comunes	básicos, internacionalización y eventos
Atributos propios	-
Tipo de elemento	En línea
Descripción	Se emplea para definir la leyenda o título de una tabla

https://developer.mozilla.org/es/docs/Trazado_de_una_tabla_HTML_mediante_JavaScript_y_la_Interface_DOM

6.5.1 Ejercicios

Ejercicio: Dada la aplicación tabla.html, se pide que al pulsar en el texto Editar de la columna Acciones, ocurra lo siguiente:

- El texto de esa columna que ponía <<Editar>> en color azul, será reemplazado por el texto <<En edición>> en color gris o negro.
- Los datos en la fila correspondiente se convertirán en casillas de texto editables de modo que el usuario pueda modificar los datos de esa fila.
- Debe aparecer en la parte inferior de la tabla el texto: <<Pulse Aceptar para guardar los cambios o cancelar para anularlos>> y dos botones: Aceptar y Cancelar, que podrán ser pulsados por el usuario para Aceptar los cambios o para cancelar.
- Si el usuario pulsa en el botón Aceptar los datos de la fila en edición deben ser enviados a una url de destino por el método get. Por ejemplo si la url de destino es aprenderaprogramar.com y editáramos la fila correspondiente a Fresa y pulsamos aceptar, el navegador debe enviarnos a una url como esta:
  http://192.168.0.1/?alimento=Fresa&calorias=57&grasas=1.5&proteina=0.4&carbohidratos=12.8
  Tener en cuenta que: los datos que se deben enviar a la url de destino son los datos editados por el usuario, no los datos originales de la tabla.
- Si el usuario pulsa en Cancelar se recargará la tabla original (de forma que no habrá ninguna fila en edición).
- Si estando seleccionada una fila en edición el usuario pulsa sobre Editar en otra fila, se mostrará un mensaje indicando lo siguiente: "Solo se puede editar una línea. Recargue la página para poder editar otra".
- La siguiente imagen refleja la idea de lo que se quiere conseguir:
  editar campos tabla javascript



- Nota: Se supone que la tabla va a cargar datos desde una base de datos, pero como nuestro objetivo no es el manejo de bases de datos ni de lenguajes del lado del servidor sino ver aplicaciones de JavaScript, por ello la tabla se os facilita con el código HTML necesario.

  Ayuda para resolver el ejercicio:

  1) Crea una variable global de nombre "editando" y tipo booleano que te permita saber si se encuentra en edición alguna fila o no.

  2) Crea una función "function transformarEnEditable(nodo)" que debe encargarse de transformar en editables los datos de una fila definiéndolos en etiquetas <input> … </input> usando innerHTML. Además debe añadir el texto "Pulse Aceptar para guardar los cambios o cancelar para anularlos" y los dos

botones: Aceptar y Cancelar. Si ya había una fila en edición, en lugar de estos se deberá mostrar el mensaje “Solo se puede editar una línea, cancele o recargue la página para poder editar otra”

3) Crea una función “function capturarEnvio()” que sirva para que cuando el usuario pulse el botón Aceptar, se cree un formulario con datos ocultos (hidden) capturados de las casillas de la fila en edición y se envíen a la url de destino por el método get. El formulario lo añadirás con innerHTML y el envío lo puedes realizar con la sentencia javascript document.nombreDelformulario.submit();

4) Crea la función anular que recargue la página original cuando el usuario pulse el botón cancelar. Esto puedes hacerlo con la sentencia JavaScript window.location.reload().

Ejercicio: Dado la siguiente aplicación y sin modificar su estructura (inicialmente se encuentra vacía), crea una tabla de 3 x 5 con los objetos del árbol DOM.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <style>
        table{
            width: 500px;
            height: 300px;}
    </style>
</head>
<body>
</body>
<script>
    //Creamos una tabla de 3 filas y 5 columnas
    //Creamos la tabla vacía:
    let miTabla = document.createElement("TABLE");
    let tblBody = document.createElement("TBODY");
    //Creamos las filas
    for(let i = 0; i<3; i++){
        let fila = document.createElement("TR");
        //Dentro de cada fila, creamos las columas
        for (let i=0; i<5; i++){
            let columna = document.createElement("TD");
            //Y cada columna creada la vinculamos a la fila
            fila.appendChild(columna);}
        //Vinculamos nuestra filas con sus celdas, al cuerpo de la tabla
        tblBody.appendChild(fila);}
    //Por último, vinculamos el tbody a la tabla, y la tabla a la página
    miTabla.appendChild(tblBody);
    document.body.appendChild(miTabla);
    miTabla.setAttribute("border", "2");
</script>
</html>
```

6.6. Ejercicio de Ampliación

Ejercicio: Realizar una aplicación Web con JavaScript que permita crear el juego de hundir la flota.

- ✓ Por un lado tendrás que crear dos tableros de 10 filas y 10 columnas.
- ✓ Crear dos jugadores (Jugador Rojo y Jugador Amarillo).
- ✓ Añadir la funcionalidad correspondiente mediante eventos para que cada jugador pueda colocar mediante un click del ratón su lista de barcos. 3 de dos posiciones, 2 de 3 posiciones y 1 de 4 posiciones.
- ✓ Además, se tendrá que comprobar mediante una función el ganador de la partida si hay barcos en horizontal, vertical y diagonal.
- ✓ Mostrar en cada jugada si: es ha tocado un barco, si ha hundido un barco o si ha sido agua.
- ✓ Finalmente, se podrá acabar la partida cuando uno de los jugadores no tenga ningún barco sobre el tablero, hasta entonces se seguirá jugando. Mostrar por pantalla el resultado de cada partida: Ganador Jugador 1 ó Ganador Jugador.